

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi • Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

VialServi

**Aplicativo web para la gestión de expedientes de servicios
mecánicos, de cerrajería, de grúa y de conductor elegido**

Documento de sustentación

Edwin Andrés Sánchez Orozco
Brahian Estiven Rendón Murillo
Juan Pablo Vásquez Marín

Facultad de Ingenierías
Politécnico Colombiano Jaime Isaza Cadavid

Proyecto Integrador
Claudia Alejandra Rosero Noguera

20 de septiembre de 2026

Contenido

1. El problema y la solución	4
1.1 Alineación con el desarrollo sostenible	4
1.2 Por qué se dice “servicios mecánicos, de cerrajería, de grúa y de conductor elegido” y no “asistencia vial”	4
2. Alcance del proyecto	5
2.1 Los módulos	5
2.2 Retención y archivado	5
2.3 Lo que este proyecto no incluye	5
2.4 Dependencia entre módulos	6
3. Objetivos	7
3.1 Objetivo general	7
3.2 Objetivos específicos	7
4. Mapa de navegación	8
5. Correcciones aplicadas tras la asesoría	10
6. Arquitectura	11
7. El expediente: ciclo de vida y reglas	12
7.1 Quién escribe qué	12
7.2 Estados del servicio	12
7.3 Un recorrido completo	12
7.4 La evidencia del cliente	13
7.5 Evidencia que llega después del cierre	13
8. El trabajo sin señal	14
8.1 El error que hay que evitar	14
8.2 El diseño: un solo camino	14
8.3 Por qué no hay conflictos	15
8.4 Reintentar no es lo mismo que un conflicto	15
8.5 Casos analizados	15
8.6 Qué cubre y qué no	16
9. Tecnologías	17
9.1 Explicación de cada tecnología	17
10. Reglas verificadas con pruebas	20
11. Estado del avance	21

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi • Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

12. Preguntas previsibles	22
13. Glosario	23
Referencias	24

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi · Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

1. El problema y la solución

VialServi presta servicios de asistencia mecánica, cerrajería, traslado en grúa y conductor elegido. Cada atención genera información que después resulta indispensable: quién atendió, en qué estado se recibió el vehículo, qué objetos había dentro, cuánto duró la atención y qué novedades se presentaron.

Hoy esa información se recoge a mano y queda dispersa: cuadernos que lleva cada técnico, fotografías en celulares personales y mensajes de WhatsApp a la central. No existe un lugar único donde consultar el historial de un servicio.

La consecuencia concreta. Cuando un cliente reclama un objeto de valor o una aseguradora pide el soporte de un servicio prestado semanas atrás, la empresa no siempre tiene con qué responder, porque las fotografías quedaron en el celular de un técnico que las borró o cambió de equipo. La empresa estima esa pérdida en cerca de \$3.200.000 mensuales entre servicios no facturables y reclamaciones sin evidencia.

La solución es un aplicativo web donde cada servicio queda registrado en un **expediente único**, alojado en línea, disponible para la empresa en cualquier momento y sin depender del dispositivo personal de nadie.

1.1 Alineación con el desarrollo sostenible

El proyecto se alinea con el Objetivo de Desarrollo Sostenible número 9, Industria, Innovación e Infraestructura, al llevar transformación digital a un sector que trabaja de forma manual. De manera complementaria aporta al Objetivo 12, Producción y Consumo Responsables, al reemplazar formularios físicos por registros digitales.

1.2 Por qué se dice “servicios mecánicos, de cerrajería, de grúa y de conductor elegido” y no “asistencia vial”

Es una precisión deliberada. “Asistencia vial” es un término que abarca desde un cambio de llanta hasta la atención de un siniestro con heridos. VialServi **no presta atención médica de emergencia** ni interviene en accidentes con lesionados. Nombrar los cuatro servicios concretos delimita el alcance y evita que se le atribuyan al proyecto responsabilidades que no tiene.

2. Alcance del proyecto

El alcance se organiza por módulos siguiendo el orden lógico en que ocurre un servicio: primero el vehículo, que es el punto de partida porque sin vehículo no hay servicio; luego el cliente al que pertenece; después el servicio que ese cliente solicita y que la central clasifica; y por último el expediente, que concentra toda la información de la atención.

2.1 Los módulos

Módulo	Qué hace
Seguridad y control de acceso	Inicio de sesión, registro, recuperación y cambio de contraseña, y administración de usuarios, roles y permisos.
Gestionar vehículo e inventario	Placa, marca, modelo, color y propietario, junto con el inventario de los objetos que quedan dentro del vehículo. Es el punto de partida del sistema.
Gestionar cliente	Datos del cliente al que pertenece el vehículo. El expediente enlaza esta información; no la duplica.
Gestionar técnico	El técnico se administra como entidad propia y no solo como rol de usuario, porque su hoja de vida —especialidades, certificaciones y licencia— determina qué servicios puede atender.
Gestionar servicio	Solicitud del cliente y clasificación por parte de la central en los tres tipos: traslado en grúa (vehículo y pasajeros), carro taller (mecánica en sitio, cambio de llantas y cerrajería) y conductor elegido, disponible 24 horas.
Gestionar expediente	Módulo central. Se abre al clasificar el servicio y concentra la información del vehículo, el cliente, el servicio y el técnico. Sobre él se diligencia el formato, se cargan evidencias, se registra el inventario y se reportan novedades.
Gestionar históricos y reportes	Almacenamiento de los expedientes cerrados, consulta del historial por placa y panel de indicadores de la operación.

2.2 Retención y archivado

Cada expediente cerrado permanece disponible de forma activa mientras la empresa lo requiera para reclamaciones o auditorías. Como este tipo de soportes suele exigirse por varios años, **el sistema no elimina información por sí solo**: los expedientes vencidos se archivan, no se borran, y la decisión de depurarlos queda en manos de la empresa.

2.3 Lo que este proyecto no incluye

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi · Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

- Atención de accidentes de tránsito con heridos o intervención de entidades de salud y de tránsito. VialServi presta asistencia mecánica, técnica y de grúa, no atención médica de emergencia.
- Funciones de chat o comunicación en tiempo real entre operarios.
- Contratación de personal, compra de equipos móviles o mejoras en la cobertura celular: son decisiones de la organización, no del software.
- Pruebas de compatibilidad con navegadores muy antiguos, pruebas de seguridad de nivel profesional, pruebas de carga sostenida y plan de recuperación ante desastres.

2.4 Dependencia entre módulos

El expediente no puede existir por sí solo: para abrirlo se necesita primero el vehículo y el cliente enlazado a él, y para cerrarlo se necesita el registro del técnico que lo atendió. Por eso el orden de construcción de los módulos sigue el mismo orden del flujo del servicio y no al contrario.

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi · Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

3. Objetivos

3.1 Objetivo general

Construir, a lo largo de las 14 semanas del Proyecto Integrador, un aplicativo web para la gestión de los servicios de carro taller (mecánica y cerrajería), traslado en grúa de vehículos y pasajeros, y conductor elegido que presta VialServi, en el que cada servicio quede registrado en un expediente único, con el formato que le corresponde y con acceso controlado por roles, desde que el cliente lo solicita hasta que la central lo cierra, con el fin de que la evidencia de cada atención esté siempre disponible para la empresa y no dependa de los dispositivos personales de los técnicos.

3.2 Objetivos específicos

Siguen las fases clásicas del ciclo de vida del desarrollo de software: análisis, diseño, construcción y pruebas.

- Analizar las necesidades de información de la empresa y los requisitos que debe cumplir el sistema, tanto en sus funciones principales como en aspectos de seguridad y desempeño.
- Diseñar la arquitectura de la plataforma y el modelo de la base de datos, dejando documentadas las decisiones tomadas y las razones detrás de cada una.
- Construir los módulos funcionales del sistema, incluyendo el registro de expedientes, la carga de evidencias, el inventario de objetos de valor y el control de acceso por roles.
- Verificar el correcto funcionamiento del sistema construido, mediante pruebas que confirmen su fiabilidad, seguridad y usabilidad conforme a los criterios de calidad de software reconocidos internacionalmente.

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi • Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

4. Mapa de navegación

El mapa representa los módulos del aplicativo y el orden en que fluye la información entre ellos. Concentra la información en un solo nivel de módulos: los roles no se ramifican, porque la asignación de permisos se resuelve en el back-end y no en la navegación. Las flechas numeradas siguen el recorrido de un servicio, desde el vehículo hasta los reportes.

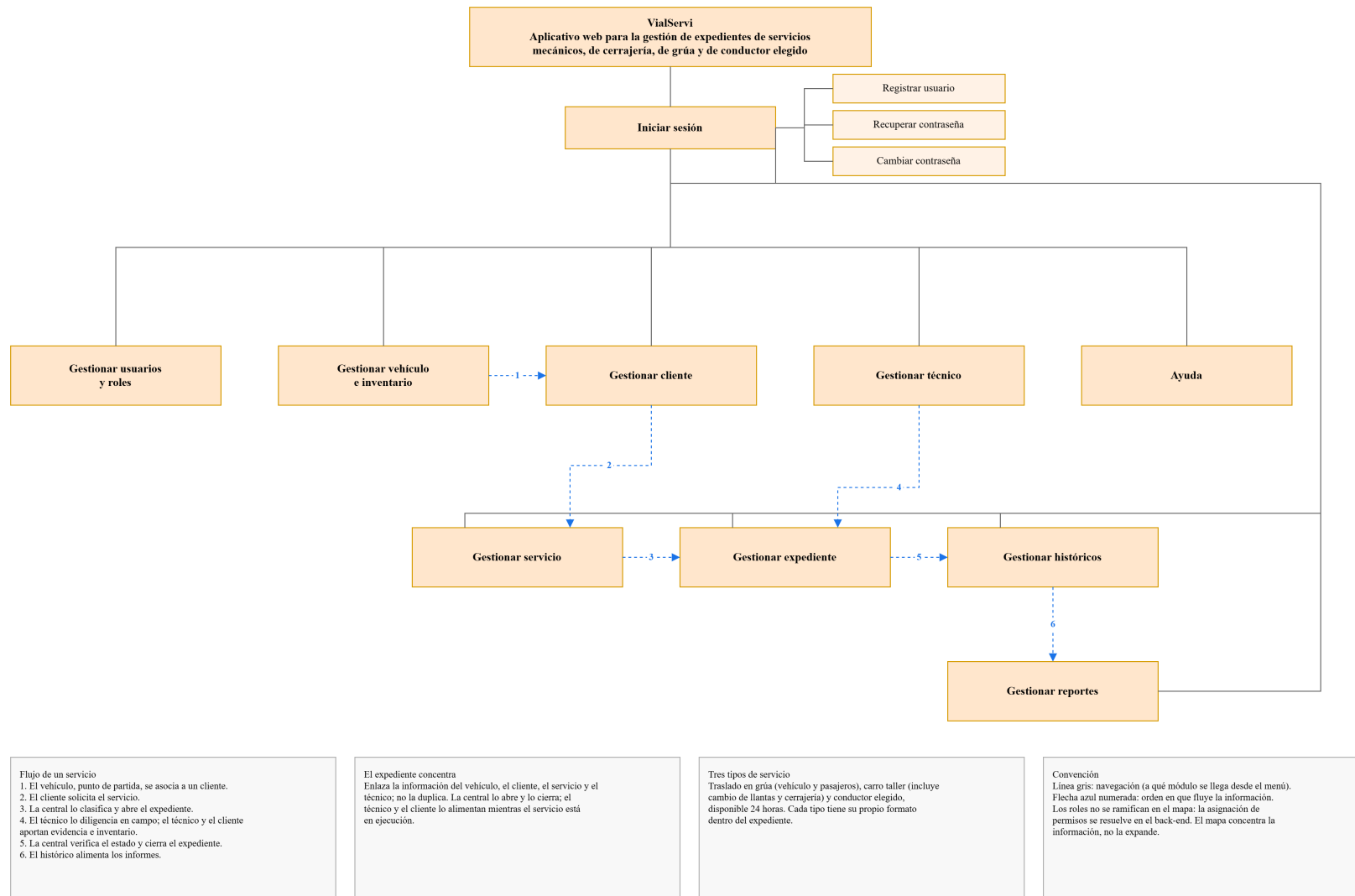


Figura 1: Mapa de navegación del aplicativo VialServi.

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi · Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

5. Correcciones aplicadas tras la asesoría

Esta sección deja explícito qué se observó, qué se cambió y dónde se puede verificar. Es la parte que conviene tener presente en la sustentación.

Observación recibida	Qué se cambió
El mapa ramificaba los roles; con muchos usuarios sería imposible de dibujar, y los permisos son asunto del back-end, no de la navegación	Los roles quedaron concentrados en un solo módulo. El mapa ya no los expande, y la convención del diagrama lo declara por escrito
El orden de los módulos no reflejaba el proceso real del negocio	El mapa sigue ahora el orden lógico: vehículo → cliente → servicio → expediente → históricos → reportes, con las relaciones numeradas de 1 a 6
El punto de partida de todo es el vehículo: sin vehículo no hay servicio	“Gestionar vehículo e inventario” es el primer módulo del flujo, y el alcance lo declara explícitamente
El técnico no es solo un rol: tiene hoja de vida con especialidades que determinan qué puede atender	“Gestionar técnico” es un módulo y una entidad propia, con especialidades y licencia. Al asignar, el sistema solo ofrece técnicos aptos y disponibles
Faltaba decir cuándo y quién cierra un servicio	Los estados quedaron definidos, y el cierre es exclusivo de la central, con condiciones verificables
“Interfaz para el técnico de campo” era una lista de requisitos no funcionales, no de funcionalidades	Ese falso módulo se eliminó del alcance. La usabilidad en celular se trata como requisito no funcional
Unificar la terminología	Lo que se llamaba “gestión de evidencias y novedades” se llama expediente en todo el documento, el código y el diagrama
“Asistencia vial” es ambiguo: sugiere atención de siniestros con heridos	Se nombran los servicios concretos en el título, el objetivo general y el mapa, y se añadió la exclusión explícita de accidentes con heridos
No es una página web, es un aplicativo web	Corregido en todo el documento, el título y el diagrama
El trabajo sin señal hay que implementarlo, no excluirlo	Salió de las exclusiones y entró al alcance, con un diseño acotado y realizable que se explica en la sección 7

6. Arquitectura

Un aplicativo web son en realidad dos programas separados que se comunican.

NAVEGADOR DEL USUARIO

SERVIDOR

```

+-----+ petición +-----+ +-----+
| Frontend | -----> | Backend | --> | Base |
| lo que se ve |      | lo que decide | | de datos |
|          | <----- | y valida | <-- |
+-----+ respuesta +-----+ +-----+

```

El **frontend** es lo que el usuario ve y toca. El **backend** es lo que decide y guarda. El punto de contacto se llama **API**: la ventanilla por la que el frontend pide y el backend responde.

Por qué importa esta separación. Las reglas del negocio viven en el backend, donde el usuario no las puede tocar. Si ocultáramos un botón en la pantalla pero la regla no estuviera en el servidor, cualquiera con conocimientos básicos podría saltársela. En la sección 9 se muestra que esto no es una afirmación: está probado.

7. El expediente: ciclo de vida y reglas

7.1 Quién escribe qué

Dato	Quién lo escribe
Solicitud: dirección, descripción, vehículo	Cliente, o Central si llamó por teléfono
Tipo de servicio y técnico asignado	Central
Consecutivo y apertura del expediente	El sistema, al clasificar
Observaciones del expediente	Técnico
Evidencias (fotografía o video de máximo 10 segundos)	Técnico y Cliente
Novedades e inventario del vehículo	Técnico
Cierre: fecha y responsable	Solo Central
Cancelación con motivo	Central
Usuarios, roles y hoja de vida del técnico	Administrador

El técnico y la central no comparten ni un solo campo del expediente. Esa es la razón por la que una actualización del técnico nunca puede sobrescribir el cierre hecho por la central. No hay conflicto porque no hay terreno compartido.

7.2 Estados del servicio

```

SOLICITADO --> ASIGNADO --> EN_EJECUCION --> TERMINADO --> CERRADO
|               |               |
+-----+-----+-----> CANCELADO

```

El técnico avanza de *asignado* a *en ejecución* y de ahí a *terminado*, porque son hechos que solo él conoce en el sitio. Solo la central pasa a *cerrado*: quien presta el servicio no declara cerrada la evidencia de que existió. Eso se llama **separación de funciones**.

La cancelación exige motivo, porque un servicio que desaparece sin explicación es justamente el hueco de trazabilidad que el sistema busca evitar. Un servicio ya cerrado no se puede cancelar.

7.3 Un recorrido completo

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi · Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

1. El cliente solicita el servicio indicando dirección, descripción y vehículo.
2. La central lo clasifica en uno de los tres tipos y lo asigna a un técnico apto. En ese momento nace el expediente con su consecutivo.
3. El técnico marca *en ejecución* al llegar al sitio.
4. Diligencia el formato del tipo de servicio, registra el inventario del vehículo, carga las evidencias y anota las novedades. El cliente puede aportar sus propias fotografías.
5. El técnico marca *terminado*.
6. La central verifica y cierra el expediente.
7. El expediente cerrado alimenta el histórico y los indicadores.

7.4 La evidencia del cliente

El cliente puede aportar sus propias fotografías al expediente de su servicio, y cada evidencia guarda **quién la subió**. Ante una reclamación por un rayón, el expediente muestra las dos versiones con su autor y su hora: las del cliente antes del traslado y las del técnico durante la atención. Sin ese dato, las fotografías serían indistinguibles y el expediente no resolvería la disputa.

7.5 Evidencia que llega después del cierre

Si una evidencia llega cuando la central ya cerró, **no se rechaza**: se guarda marcada como posterior al cierre. El criterio es que perder evidencia es peor que tener un registro tardío, y en una reclamación esa fotografía puede ser justamente la que importa.

8. El trabajo sin señal

El técnico atiende en carretera, donde la cobertura falla. El aplicativo debe permitirle registrar sin conexión y enviar después.

8.1 El error que hay que evitar

La forma intuitiva es construir dos modos: uno conectado que guarda en el servidor y otro desconectado que guarda en el celular. Eso obliga a escribir la misma lógica dos veces y, al reconectar, a comparar dos versiones de la verdad y decidir cuál gana. Ahí es donde este tipo de solución se vuelve inabordable.

8.2 El diseño: un solo camino

```

El técnico pulsa "Guardar"
      |
      v
+-----+
| BUZÓN DE SALIDA | Siempre se escribe aquí primero,
| (IndexedDB)     | haya señal o no. Sin excepción.
+-----+
      |
      ¿hay conexión?
      |           |
      SÍ          NO
      |           |
      v           v
se envía      queda pendiente
al API        ("2 sin enviar")
      |           |
      |           +--> al volver la señal
      |           se reintenta solo
      v
el servidor confirma --> se borra del buzón

```

Con señal ese recorrido dura milisegundos y el técnico no nota que el buzón existe. Sin señal, el registro espera. **Es el mismo código en ambos casos.**

8.3 Por qué no hay conflictos

1. **Casi todo lo que hace el técnico son inserciones, no ediciones.** Una fotografía, una novedad, un objeto del inventario: cada uno es una fila nueva. Dos filas nuevas jamás se pisan.
2. **Los campos están repartidos por rol**, como se mostró en la sección anterior.
3. **La sincronización va en un solo sentido**, del dispositivo al servidor. No se bajan cambios para fusionarlos, de modo que la pregunta “¿cuál versión gana?” no llega a plantearse.

8.4 Reintentar no es lo mismo que un conflicto

Cada registro viaja con un identificador generado en el dispositivo. Si la petición se reintentada diez veces, el servidor reconoce que ya tiene ese registro y no lo duplica. Para el único campo que el técnico edita —las observaciones— existe además un contador de versión: el servidor descarta cualquier actualización cuyo contador sea menor al guardado, de modo que un envío viejo que llega tarde no pisa el texto nuevo.

Se usa un contador y no la hora del dispositivo porque un reloj mal configurado podría revertir una edición más reciente.

8.5 Casos analizados

Caso	Comportamiento
Registro reintentado muchas veces	El identificador del dispositivo hace que se aplique una sola vez
Expediente creado sin señal y evidencias de ese expediente	Las evidencias referencian al padre por su identificador local; la cola envía el padre primero
Evidencia que llega tras el cierre	Se acepta y se marca
Registro inválido que el servidor rechaza siempre	Tras varios intentos se aparta como “requiere revisión”; la cola no se bloquea
Se cierra el navegador o se agota la batería	El buzón vive en la base del navegador, no en memoria: al reabrir sigue ahí
Dos pestañas sincronizando a la vez	El identificador único descarta el duplicado
Reloj del dispositivo mal configurado	Para las observaciones manda el contador, no la hora
Orden de llegada de las evidencias	Se muestran por la hora en que se tomaron, no por la de llegada

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi · Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

8.6 Qué cubre y qué no

Cubre el registro en campo: diligenciar el formato, tomar fotografías, grabar clips de máximo diez segundos y anotar novedades sin cobertura. **No cubre** la consulta del histórico completo sin conexión, que sí requiere red.

9. Tecnologías

Todas tienen licencia libre o un plan gratuito suficiente para el proyecto, de modo que su implantación no depende de un presupuesto.

Capa	Tecnología	Para qué se usa
Lenguaje	TypeScript	Único lenguaje del proyecto, en servidor e interfaz
Servidor	Node.js con Express	API por la que se exponen los módulos
Validación	Zod	Revisa los datos antes de que lleguen a la base
Base de datos	Prisma ORM sobre SQLite, portable a PostgreSQL	Esquema, migraciones versionadas y consultas
Seguridad	JWT y bcrypt	Sesiones con vencimiento y contraseñas cifradas
Interfaz	React con Vite	Pantallas del aplicativo
Estilos	Tailwind CSS	Diseño, incluida la vista del técnico en celular
Estado y datos	TanStack Query	Consulta al API con caché y manejo de errores
Pruebas	Vitest y Supertest	Pruebas automatizadas del API
Pruebas manuales	Postman	Prueba de cada ruta durante el desarrollo
Nube	Supabase	PostgreSQL gestionado y almacenamiento de fotografías
Sin señal	Dexie.js sobre IndexedDB	Buzón de salida en el dispositivo
Instalable	vite-plugin-pwa	Permite que el aplicativo abra sin señal
Versiones	Git	Historial del código y la documentación
Ejecución	Node.js 20 y npm workspaces	Servidor e interfaz en un repositorio

9.1 Explicación de cada tecnología

TypeScript. Es JavaScript con verificación de tipos: revisa el código antes de ejecutarlo. Si se escribe mal el nombre de un campo, el editor lo marca en rojo en vez de que el error aparezca cuando el usuario ya está usando el sistema. Al ser el mismo lenguaje en los dos programas, un cambio en el modelo de datos se propaga y el editor señala todos los sitios que quedaron mal.

Node.js. Permite que JavaScript funcione fuera del navegador, como un programa del computador. Sin él, JavaScript solo serviría para páginas web.

Express. Es el recepcionista del backend: mira a qué dirección va cada petición y la envía a la función que corresponde.

Zod. Es el control de entrada del API. Por ejemplo, exige que el motivo de una cancelación tenga al menos cinco caracteres; si llega vacío, la petición se rechaza y la base de datos ni se entera.

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi · Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

Está en el servidor, así que aunque alguien evite la interfaz, la validación se aplica igual.

Prisma (un ORM). ORM significa *mapeo objeto-relacional* y es un traductor: las bases de datos hablan SQL y los programas hablan en objetos. Aporta tres cosas: *migraciones* —cada cambio de estructura queda como archivo versionado, de modo que todos los integrantes tienen la misma base—; *portabilidad* —el mismo código corre sobre SQLite y sobre PostgreSQL—; y *tipos automáticos* generados desde el esquema.

SQLite y PostgreSQL. Son dos motores de base de datos. SQLite guarda todo en un archivo y no requiere instalación, ideal para desarrollar. PostgreSQL es un motor completo para cuando el sistema esté en producción.

Supabase. Servicio que entrega PostgreSQL alojado en internet más almacenamiento de archivos. Las fotografías no deben guardarse dentro de la base —la vuelven lenta y difícil de respaldar—: van al almacenamiento y la base guarda la ruta.

bcrypt. Un *hash* es una transformación de una sola vía: de la contraseña sale un texto ininteligible del que no se puede volver atrás. Al iniciar sesión no se comparan contraseñas, se comparan las transformaciones. Si alguien viera la base de datos, no obtendría la contraseña de nadie.

JWT. Un *token* funciona como la manilla de un concierto: se muestra el documento una sola vez al entrar y después basta enseñar la manilla, que el portero reconoce por el sello. El servidor no tiene que recordar quién está conectado, y la firma impide que alguien se fabrique un token diciendo que es administrador. Los del proyecto vencen a las ocho horas.

React. Construye la interfaz con componentes reutilizables y actualiza solo lo que cambió, en vez de redibujar la pantalla entera.

Vite. Herramienta de construcción. Mientras se programa, actualiza la pantalla al instante al guardar un archivo; para producción, empaqueta todo en pocos archivos comprimidos. En este proyecto la diferencia medida fue de 21 peticiones en desarrollo a 3 en producción.

Tailwind CSS. Los estilos se escriben como clases pequeñas sobre el elemento en vez de en un archivo aparte, de modo que se ve al instante qué estilo tiene cada cosa.

TanStack Query. Administra las consultas al servidor desde la interfaz: guarda en caché lo ya pedido, evita peticiones repetidas y maneja los estados de carga y error.

IndexedDB y Dexie.js. IndexedDB es una base de datos dentro del navegador, que no se borra al cerrar la pestaña. Dexie es una capa que la hace manejable. Ahí vive el buzón de salida.

Service worker y PWA. Un *service worker* es un pequeño programa que se sitúa entre la aplicación y la red y puede responder con copias guardadas. Es lo que permite que el aplicativo abra sin señal; sin él, el navegador intentaría descargar la página y mostraría error.

Vitest y Supertest. Vitest ejecuta las pruebas; Supertest hace peticiones al API sin levantar un servidor real, por eso las 26 pruebas corren en tres segundos.

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi · Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

Git y npm workspaces. Git guarda el historial del proyecto. Un *monorepo* es un repositorio con varios proyectos dentro: aquí conviven el servidor y la interfaz, de modo que un cambio que afecta a ambos va en el mismo registro y no pueden desincronizarse.

10. Reglas verificadas con pruebas

Las reglas no se afirman: se demuestran. El proyecto tiene **26 pruebas automatizadas** que se ejecutan con un comando y verifican el API directamente, sin pasar por la interfaz. Todas pasan.

Lo que se comprueba	Resultado
Sin sesión no se entra a ningún módulo	Rechazado (401)
Una clave incorrecta no revela si el documento existe	Mismo mensaje
El técnico solo recibe sus servicios; el cliente, los suyos	Verificado
El técnico no puede consultar los reportes	Prohibido (403)
El técnico no puede cerrar, aunque llame la ruta directamente	Prohibido (403)
El técnico no puede clasificar ni asignar	Prohibido (403)
No se cierra un servicio que no esté terminado	Rechazado (409)
No se cierra un expediente sin evidencias	Rechazado (409)
Un expediente cerrado no se cierra dos veces	Rechazado (409)
Cancelar sin motivo	Rechazado (400)
Un servicio cerrado no se puede cancelar	Rechazado (409)
La misma evidencia enviada tres veces	Se guarda una sola
Una actualización vieja que llega tarde	No pisa la nueva
Evidencia sobre expediente cerrado	Se acepta y se marca
La evidencia guarda quién la aportó	Verificado
Al clasificar nace el expediente con consecutivo único	Verificado
Los indicadores cuadran con la base de datos	Verificado

Por qué esto importa. La prueba del técnico que llama directamente la ruta de cierre y recibe un 403 demuestra que la regla vive en el servidor y no en esconder un botón. Es la diferencia entre una restricción real y una apariencia.

11. Estado del avance

Módulo	Estado
Seguridad y control de acceso	Construido
Gestionar vehículo e inventario	Construido
Gestionar cliente	Construido
Gestionar técnico	Construido (consulta)
Gestionar servicio	Construido: solicitar, clasificar, asignar, avanzar estado y cancelar
Gestionar expediente	Construido: observaciones, evidencias, novedades y cierre
Gestionar históricos	Construido: consulta por placa
Gestionar reportes	Construido: panel de indicadores
Gestionar usuarios y roles	Pendiente
Trabajo sin señal	Servidor preparado y probado; falta el buzón en el dispositivo
Carga real de archivos	Pendiente: hoy se registra la ruta de la evidencia

Una precisión sobre el trabajo sin señal. La parte difícil ya está hecha y probada: que el servidor tolere reintentos sin duplicar, descarte actualizaciones que llegan tarde y marque la evidencia posterior al cierre. Lo que falta es la mecánica del dispositivo, que sobre esa base es considerablemente más simple.

Los módulos pendientes aparecen marcados como tales en el menú del aplicativo, y el API responde explícitamente que no están implementados, de modo que la navegación y el código dicen lo mismo.

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi · Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

12. Preguntas previsibles

¿Por qué el técnico no puede cerrar el expediente que él mismo atendió? Por separación de funciones. Quien presta el servicio no debería poder declarar cerrada —ni eliminar— la evidencia de que existió. Es la misma lógica por la que quien ejecuta un gasto no lo aprueba.

¿Qué pasa si el técnico no tiene señal? Registra igual. El formulario guarda primero en el dispositivo y envía después. Se puede demostrar en vivo desactivando la red desde el navegador.

¿Y si sube la evidencia dos veces por reintentos? No se duplica: cada registro lleva un identificador generado en el dispositivo y el servidor lo reconoce.

¿Por qué el video está limitado a diez segundos? Por el almacenamiento. Un clip de celular pesa decenas de megabytes; el plan gratuito da un gigabyte, que con video libre se agota en pocos servicios. Las fotografías cubren la necesidad probatoria, que es de lo que se trata el expediente.

¿Por qué SQLite y no un motor “de verdad”? SQLite es para desarrollar: no requiere instalación y cada integrante tiene su base al clonar el proyecto. Como se usa un ORM con migraciones, el paso a PostgreSQL sobre Supabase es un cambio de configuración, no una reescritura.

¿Cómo se sabe que los permisos funcionan de verdad? Hay pruebas que llaman las rutas prohibidas directamente, sin pasar por la interfaz, y comprueban que el servidor responde 403.

¿Qué pasa si el cliente y el técnico dicen cosas distintas sobre un rayón? El expediente guarda ambas evidencias con su autor y su hora. No decide quién tiene razón: aporta el soporte para que la empresa lo resuelva, que es justamente lo que hoy no tiene.

¿Por qué el técnico se gestiona aparte y no como un rol más? Porque tiene hoja de vida: especialidades, certificaciones y licencia determinan qué servicios puede atender. Un rol es un permiso; esto es información del negocio.

¿El sistema está en producción? No. Es un avance funcional y verificado. El despliegue requiere la decisión de la empresa y completar los módulos pendientes.

13. Glosario

Término	Significado
Frontend	Lo que el usuario ve y toca, en su navegador
Backend	El programa del servidor: decide, valida y guarda
API	La ventanilla por la que frontend y backend se comunican
Ruta o <i>endpoint</i>	Una dirección concreta del API
ORM	Traductor entre los objetos del programa y las tablas de la base
Migración	Un cambio en la estructura de la base, guardado como archivo versionado
Hash	Transformación de una sola vía; se usa para las contraseñas
Token	Credencial firmada que prueba quién es el usuario en cada petición
Caché	Copia guardada para no volver a pedir lo mismo
Componente	Pieza reutilizable de interfaz
Monorepo	Un repositorio que contiene varios proyectos
PWA	Aplicativo web que se instala y abre sin conexión
Service worker	Programa que se sitúa entre la aplicación y la red
Idempotencia	Que repetir una operación produzca el mismo resultado que hacerla una vez

 POLITÉCNICO COLOMBIANO JAIME ISAZA CADAVID	PROYECTO INTEGRADOR VialServi • Documento de sustentación	Código: PI-2026-2 Versión: 01
--	--	--

Referencias

- Congreso de la República de Colombia. (2012). *Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. <https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981>
- International Organization for Standardization. (2011). *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*.
- Organización de las Naciones Unidas. (2015). *Objetivos de Desarrollo Sostenible*. <https://www.un.org/sustainabledevelopment/es/>
- Pressman, R. (2010). *Ingeniería de software: un enfoque práctico* (7.^a ed.). McGraw Hill.
- Sommerville, I. (2005). *Ingeniería del software* (7.^a ed.). McGraw Hill.